Pandas is a Python library.

Pandas is used to analyze data.

Panda Intro

What is pandas ?

Pandas is a Python library used for working with data sets.

It has functions for analyzing, cleaning, exploring, and manipulating data.

The name "Pandas" has a reference to both "Panel Data", and "Python Data Analysis" and was created by Wes McKinney in 2008.

Why use pandas ?

Pandas allows us to analyze big data and make conclusions based on statistical theories.

Pandas can clean messy data sets, and make them readable and relevant.

Relevant data is very important in data science.

:} **Data Science:** is a branch of computer science where we study how to store, use and analyze data for deriving information from it.

What Can Pandas Do ?

Pandas gives you answers about the data. Like:

Is there a correlation between two or more columns? What is average value? Max value? Min value?

Pandas Series

series is like a column in a table

In [3]:
```
import pandas as pd
arr=[1,2,3]
sr=pd.Series(arr)
print(sr)
```

```
0    1
1    2
2    3
dtype: int64
```

adding labels to the series

In [4]:
```
sr=pd.Series(arr,index=["lab1","lab2","lab3"])
print(sr)
```

```
lab1    1
lab2    2
lab3    3
dtype: int64
```

In [5]:
```
import pandas as pd

calories = {"day1": 420, "day2": 380, "day3": 390}

myvar = pd.Series(calories)

print(myvar)
```

```
day1    420
day2    380
day3    390
dtype: int64
```

Pandas DataFrames

In [6]:
```
import pandas as pd
df1=['hamza','bilal','azaan']
df=pd.DataFrame(df1)
df
```

Out[6]:

	0
0	hamza
1	bilal
2	azaan

In [8]:
```
import pandas as pd
data={
   'Name':["hamza","umer","jabbar"],
   'CGPA':[2.42,2.28,3.8]
}
df2=pd.DataFrame(data)
df2
```

Out[8]:

	Name	CGPA
0	hamza	2.42
1	umer	2.28
2	jabbar	3.80

as we can see above,the dataframe is like an table with rows and columns

Pandas use the **loc** attribute to return one or more specified row(s)

In [9]:
```
#refer to the row index:
print(df2.loc[0])
```

```
Name    hamza
CGPA     2.42
Name: 0, dtype: object
```

In [10]:
```
print(df2.loc[[0, 1]])
```

```
    Name  CGPA
0  hamza  2.42
1   umer  2.28
```

we can also name our indexes

In [12]:
```
df2=pd.DataFrame(data,index=["1669","1677","1845"])
print(df2)
```

```
        Name  CGPA
1669   hamza  2.42
1677    umer  2.28
1845  jabbar  3.80
```

In [13]:
```
print(df2.loc["1677"])
```

```
Name    umer
CGPA    2.28
Name: 1677, dtype: object
```

Reading CSV files

A simple way to store big data sets is to use CSV files (comma separated files).

CSV files contains plain text and is a well know format that can be read by everyone including Pandas.

In [14]:
```
# make sure you have your your csv file in same folder as this code file,otherwise you have to provid
import pandas as pd
data2=pd.read_csv('heart.csv')
data2
```

Out[14]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
0	52	1	0	125	212	0	1	168	0	1.0	2	2	3	0
1	53	1	0	140	203	1	0	155	1	3.1	0	0	3	0
2	70	1	0	145	174	0	1	125	1	2.6	0	0	3	0
3	61	1	0	148	203	0	1	161	0	0.0	2	1	3	0
4	62	0	0	138	294	1	1	106	0	1.9	1	3	2	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
1020	59	1	1	140	221	0	1	164	1	0.0	2	0	2	1
1021	60	1	0	125	258	0	0	141	1	2.8	1	1	3	0
1022	47	1	0	110	275	0	0	118	1	1.0	1	1	2	0
1023	50	0	0	110	254	0	0	159	0	0.0	2	0	2	1
1024	54	1	0	120	188	0	1	113	0	1.4	1	1	3	0

1025 rows × 14 columns

if you have a large DataFrame with many rows, Pandas will only return the first 5 rows, and the last 5 rows,like it did above

Analyzing Data using pandas

we use our same dataset above

The **head()** method returns the headers and a specified number of rows, starting from the top.

In [17]:
```
data2.head()# by deafult gives 5,we can add value to get how many we want
```

Out[17]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
0	52	1	0	125	212	0	1	168	0	1.0	2	2	3	0
1	53	1	0	140	203	1	0	155	1	3.1	0	0	3	0
2	70	1	0	145	174	0	1	125	1	2.6	0	0	3	0
3	61	1	0	148	203	0	1	161	0	0.0	2	1	3	0
4	62	0	0	138	294	1	1	106	0	1.9	1	3	2	0

In [18]:
```
data2.head(10)
```

Out[18]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
0	52	1	0	125	212	0	1	168	0	1.0	2	2	3	0
1	53	1	0	140	203	1	0	155	1	3.1	0	0	3	0
2	70	1	0	145	174	0	1	125	1	2.6	0	0	3	0
3	61	1	0	148	203	0	1	161	0	0.0	2	1	3	0
4	62	0	0	138	294	1	1	106	0	1.9	1	3	2	0
5	58	0	0	100	248	0	0	122	0	1.0	1	0	2	1
6	58	1	0	114	318	0	2	140	0	4.4	0	3	1	0
7	55	1	0	160	289	0	0	145	1	0.8	1	1	3	0
8	46	1	0	120	249	0	0	144	0	0.8	2	0	3	0
9	54	1	0	122	286	0	0	116	1	3.2	1	2	2	0

There is also a **tail()** method for viewing the last rows of the DataFrame.

In [19]:
```
data2.tail()
```

Out[19]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
1020	59	1	1	140	221	0	1	164	1	0.0	2	0	2	1
1021	60	1	0	125	258	0	0	141	1	2.8	1	1	3	0
1022	47	1	0	110	275	0	0	118	1	1.0	1	1	2	0
1023	50	0	0	110	254	0	0	159	0	0.0	2	0	2	1
1024	54	1	0	120	188	0	1	113	0	1.4	1	1	3	0

The DataFrames object has a method called **info()**, that gives you more information about the data set.

In [20]:
```
data2.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1025 entries, 0 to 1024
Data columns (total 14 columns):
 #   Column    Non-Null Count  Dtype  
---  ------    --------------  -----  
 0   age       1025 non-null   int64  
 1   sex       1025 non-null   int64  
 2   cp        1025 non-null   int64  
 3   trestbps  1025 non-null   int64  
 4   chol      1025 non-null   int64  
 5   fbs       1025 non-null   int64  
 6   restecg   1025 non-null   int64  
 7   thalach   1025 non-null   int64  
 8   exang     1025 non-null   int64  
 9   oldpeak   1025 non-null   float64
 10  slope     1025 non-null   int64  
 11  ca        1025 non-null   int64  
 12  thal      1025 non-null   int64  
 13  target    1025 non-null   int64  
dtypes: float64(1), int64(13)
memory usage: 112.2 KB
```

dealing with COLUMNS

In [21]:
```
# renaming columns
```

In [23]:
```
data2.columns# before renaming
```

Out[23]:
```
Index(['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg', 'thalach',
       'exang', 'oldpeak', 'slope', 'ca', 'thal', 'target'],
      dtype='object')
```

In [24]:
```
data2.columns=['age','gender','chest_pain_type','resting_bp','cholestrol','fasting_blood_sugar', 'res
       'exercise_induced_angina', 'st_depression', 'st_slope', 'num_major_vessels', 'thalassemia', 'targ

]
data2.head()# changed column names
```

Out[24]:

	age	gender	chest_pain_type	resting_bp	cholestrol	fasting_blood_sugar	rest_ecg	max_heart_r
0	52	1	0	125	212	0	1	
1	53	1	0	140	203	1	0	
2	70	1	0	145	174	0	1	
3	61	1	0	148	203	0	1	
4	62	0	0	138	294	1	1	

In [25]:
```
# renaming specific column
```

In [26]:
```
d1=data2.rename(columns={'rest_ecg':'resting electrocardiographic result'})
d1.head()
```

Out[26]:

	age	gender	chest_pain_type	resting_bp	cholestrol	fasting_blood_sugar	resting electrocardiographic result
0	52	1	0	125	212	0	1
1	53	1	0	140	203	1	0
2	70	1	0	145	174	0	1
3	61	1	0	148	203	0	1
4	62	0	0	138	294	1	1

In [27]:
```
# adding columns
```

In [30]:
```
d2=data2.head(10)
names = ['Ali', 'Salman', 'Sohail', 'Mohsin','Waqas','Zeshan','Babar','John','Elon','Michael']
d2.insert(0,'name',names)
d2
```

Out[30]:

	name	age	gender	chest_pain_type	resting_bp	cholestrol	fasting_blood_sugar	rest_ecg	ma
0	Ali	52	1	0	125	212	0	1	
1	Salman	53	1	0	140	203	1	0	
2	Sohail	70	1	0	145	174	0	1	
3	Mohsin	61	1	0	148	203	0	1	
4	Waqas	62	0	0	138	294	1	1	
5	Zeshan	58	0	0	100	248	0	0	
6	Babar	58	1	0	114	318	0	2	
7	John	55	1	0	160	289	0	0	
8	Elon	46	1	0	120	249	0	0	
9	Michael	54	1	0	122	286	0	0	

In [31]:
```
#droping columns
#df. drop("A", axis = 1) , it will drop a whole column.
#"axis 0" represents rows
```

In [34]:
```
data2.drop(['rest_ecg'], axis = 1, inplace = True)
data2
```

Out[34]:

	age	gender	chest_pain_type	resting_bp	cholestrol	fasting_blood_sugar	max_heart_rate_ac
0	52	1	0	125	212	0	
1	53	1	0	140	203	1	
2	70	1	0	145	174	0	
3	61	1	0	148	203	0	
4	62	0	0	138	294	1	
...	...	...	...	...	...	...	
1020	59	1	1	140	221	0	
1021	60	1	0	125	258	0	
1022	47	1	0	110	275	0	
1023	50	0	0	110	254	0	
1024	54	1	0	120	188	0	

1025 rows × 13 columns

dealing with ROWS

In [35]:
```
import pandas as pd
data3=pd.read_csv('diabetes.csv')
data3.head(10)
```

Out[35]:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	A
0	6	148	72	35	0	33.6	0.627	
1	1	85	66	29	0	26.6	0.351	
2	8	183	64	0	0	23.3	0.672	
3	1	89	66	23	94	28.1	0.167	
4	0	137	40	35	168	43.1	2.288	
5	5	116	74	0	0	25.6	0.201	
6	3	78	50	32	88	31.0	0.248	
7	10	115	0	0	0	35.3	0.134	
8	2	197	70	45	543	30.5	0.158	
9	8	125	96	0	0	0.0	0.232	

In [36]:
```
# adding row
```

In [41]:
```
new_rows={'Pregnancies':7,'Glucose':190,'BloodPressure':99,'SkinThickness':12,'Insulin':0,'Diabetes
        'Age':42,'Outcome':1}
#ignore_index by-default=False. If True, index values are not used along concatenation axis and the
#Cannot append dictionary if ignore_index=False
data3=data3.append(new_rows,ignore_index=True)
data3
```

Out[41]:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction
0	9.0	140.0	71.0	32.0	1.0	26.5	0.005
1	1.0	85.0	66.0	29.0	0.0	26.6	0.351
2	8.0	183.0	64.0	0.0	0.0	23.3	0.672
3	1.0	89.0	66.0	23.0	94.0	28.1	0.167
4	0.0	137.0	40.0	35.0	168.0	43.1	2.288
...	...	...	...	...	...	...	...
106	1.0	96.0	122.0	0.0	0.0	22.4	0.207
107	4.0	144.0	58.0	28.0	140.0	29.5	0.287
108	7.0	190.0	99.0	NaN	0.0	NaN	0.962
109	7.0	190.0	99.0	NaN	0.0	NaN	0.962
110	7.0	190.0	99.0	12.0	0.0	NaN	0.962

111 rows × 10 columns

In [39]:
```
# add row at specific index using loc
```

In [42]:
```
data3.loc[0] = [9,140,71,32,1,26.5,0.005,63,1,13]
data3
```

Out[42]:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction
0	9.0	140.0	71.0	32.0	1.0	26.5	0.005
1	1.0	85.0	66.0	29.0	0.0	26.6	0.351
2	8.0	183.0	64.0	0.0	0.0	23.3	0.672
3	1.0	89.0	66.0	23.0	94.0	28.1	0.167
4	0.0	137.0	40.0	35.0	168.0	43.1	2.288
...	...	...	...	...	...	...	...
106	1.0	96.0	122.0	0.0	0.0	22.4	0.207
107	4.0	144.0	58.0	28.0	140.0	29.5	0.287
108	7.0	190.0	99.0	NaN	0.0	NaN	0.962
109	7.0	190.0	99.0	NaN	0.0	NaN	0.962
110	7.0	190.0	99.0	12.0	0.0	NaN	0.962

111 rows × 10 columns

In [44]:
```
#select rows faster
data3.iloc[0] # first row of data frame (

# data3.iloc[-1] # last row of data frame
```

Out[44]:
```
Pregnancies                 9.000
Glucose                   140.000
BloodPressure              71.000
SkinThickness              32.000
Insulin                     1.000
BMI                        26.500
DiabetesPedigreeFunction    0.005
Age                        63.000
Outcome                     1.000
Skin Thickness             13.000
Name: 0, dtype: float64
```

In [45]:
```
data3.iloc[1] # second row of data frame
```

Out[45]:
```
Pregnancies                 1.000
Glucose                    85.000
BloodPressure              66.000
SkinThickness              29.000
Insulin                     0.000
BMI                        26.600
DiabetesPedigreeFunction    0.351
Age                        31.000
Outcome                     0.000
Skin Thickness                NaN
Name: 1, dtype: float64
```

In [46]:
```
data3.iloc[-1] # last row of data frame
```

Out[46]:
```
Pregnancies                 7.000
Glucose                   190.000
BloodPressure              99.000
SkinThickness              12.000
Insulin                     0.000
BMI                           NaN
DiabetesPedigreeFunction    0.962
Age                        42.000
Outcome                     1.000
Skin Thickness                NaN
Name: 110, dtype: float64
```

Some more Setting

In [47]:
```
import pandas as pd
data4=pd.read_csv('winequality-red.csv')
data4.head()
```

Out[47]:

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol
0	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4
1	7.8	0.88	0.00	2.6	0.098	25.0	67.0	0.9968	3.20	0.68	9.8
2	7.8	0.76	0.04	2.3	0.092	15.0	54.0	0.9970	3.26	0.65	9.8
3	11.2	0.28	0.56	1.9	0.075	17.0	60.0	0.9980	3.16	0.58	9.8
4	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4

In [49]:
```
data4[['fixed acidity','citric acid']]
```

Out[49]:

	fixed acidity	citric acid
0	7.4	0.00
1	7.8	0.00
2	7.8	0.04
3	11.2	0.56
4	7.4	0.00
...	...	...
1594	6.2	0.08
1595	5.9	0.10
1596	6.3	0.13
1597	5.9	0.12
1598	6.0	0.47

1599 rows × 2 columns

In [50]:
```
# setting the name of indexex
```

In [52]:
```
data4=data4.rename_axis('vinesample')
data4
```

Out[52]:

vinesample	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates
0	7.4	0.700	0.00	1.9	0.076	11.0	34.0	0.99780	3.51	0.56
1	7.8	0.880	0.00	2.6	0.098	25.0	67.0	0.99680	3.20	0.68
2	7.8	0.760	0.04	2.3	0.092	15.0	54.0	0.99700	3.26	0.65
3	11.2	0.280	0.56	1.9	0.075	17.0	60.0	0.99800	3.16	0.58
4	7.4	0.700	0.00	1.9	0.076	11.0	34.0	0.99780	3.51	0.56
...	...	...	...	...	...	...	...	...	...	...
1594	6.2	0.600	0.08	2.0	0.090	32.0	44.0	0.99490	3.45	0.58
1595	5.9	0.550	0.10	2.2	0.062	39.0	51.0	0.99512	3.52	0.76
1596	6.3	0.510	0.13	2.3	0.076	29.0	40.0	0.99574	3.42	0.75
1597	5.9	0.645	0.12	2.0	0.075	32.0	44.0	0.99547	3.57	0.71
1598	6.0	0.310	0.47	3.6	0.067	18.0	42.0	0.99549	3.39	0.66

1599 rows × 12 columns

In [53]:
```
data4.iloc[0][0]# gives values for columns like value of first column of firstrow
```

Out[53]:
```
7.4
```

In [54]:
```
# we can sort values for specific columns
```

In [55]:
```
data4.sort_values('fixed acidity',inplace=True)
data4
```

Out[55]:

vinesample	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates
45	4.6	0.520	0.15	2.1	0.054	8.0	65.0	0.99340	3.90	0.56
95	4.7	0.600	0.17	2.3	0.058	17.0	106.0	0.99320	3.85	0.60
821	4.9	0.420	0.00	2.1	0.048	16.0	42.0	0.99154	3.71	0.74
588	5.0	0.420	0.24	2.0	0.060	19.0	50.0	0.99170	3.72	0.74
94	5.0	1.020	0.04	1.4	0.045	41.0	85.0	0.99380	3.75	0.48
...	...	...	...	...	...	...	...	...	...	...
555	15.5	0.645	0.49	4.2	0.095	10.0	23.0	1.00315	2.92	0.74
554	15.5	0.645	0.49	4.2	0.095	10.0	23.0	1.00315	2.92	0.74
442	15.6	0.685	0.76	3.7	0.100	6.0	43.0	1.00320	2.95	0.68
557	15.6	0.645	0.49	4.2	0.095	10.0	23.0	1.00315	2.92	0.74
652	15.9	0.360	0.65	7.5	0.096	22.0	71.0	0.99760	2.98	0.84

1599 rows × 12 columns

In [56]:
```
# change values
```

In [58]:
```
data4.at[45,'fixed acidity'] = 4.2# change values
# 45 shows index number
data4
```

Out[58]:

vinesample	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates
45.0	4.2	0.520	0.15	2.1	0.054	8.0	65.0	0.99340	3.90	0.56
95.0	4.7	0.600	0.17	2.3	0.058	17.0	106.0	0.99320	3.85	0.60
821.0	4.9	0.420	0.00	2.1	0.048	16.0	42.0	0.99154	3.71	0.74
588.0	5.0	0.420	0.24	2.0	0.060	19.0	50.0	0.99170	3.72	0.74
94.0	5.0	1.020	0.04	1.4	0.045	41.0	85.0	0.99380	3.75	0.48
...	...	...	...	...	...	...	...	...	...	...
554.0	15.5	0.645	0.49	4.2	0.095	10.0	23.0	1.00315	2.92	0.74
442.0	15.6	0.685	0.76	3.7	0.100	6.0	43.0	1.00320	2.95	0.68
557.0	15.6	0.645	0.49	4.2	0.095	10.0	23.0	1.00315	2.92	0.74
652.0	15.9	0.360	0.65	7.5	0.096	22.0	71.0	0.99760	2.98	0.84
4.6	4.2	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN

1600 rows × 12 columns

concatenation

In [59]:
```
# concatenating dataset3,4
res=pd.concat([data3,data4],ignore_index=True)
dr1=res.head(10)
dr1
```

Out[59]:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	A
0	9.0	140.0	71.0	32.0	1.0	26.5	0.005	6
1	1.0	85.0	66.0	29.0	0.0	26.6	0.351	3
2	8.0	183.0	64.0	0.0	0.0	23.3	0.672	3
3	1.0	89.0	66.0	23.0	94.0	28.1	0.167	2
4	0.0	137.0	40.0	35.0	168.0	43.1	2.288	3
5	5.0	116.0	74.0	0.0	0.0	25.6	0.201	3
6	3.0	78.0	50.0	32.0	88.0	31.0	0.248	2
7	10.0	115.0	0.0	0.0	0.0	35.3	0.134	2
8	2.0	197.0	70.0	45.0	543.0	30.5	0.158	5
9	8.0	125.0	96.0	0.0	0.0	0.0	0.232	5

10 rows × 22 columns

In [60]:
```
# concat columns

res2=pd.concat([res,data4],axis=1,ignore_index=False)
dr2=res2.head()
dr2
```

Out[60]:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction
0.0	9.0	140.0	71.0	32.0	1.0	26.5	0.005
1.0	1.0	85.0	66.0	29.0	0.0	26.6	0.351
2.0	8.0	183.0	64.0	0.0	0.0	23.3	0.672
3.0	1.0	89.0	66.0	23.0	94.0	28.1	0.167
4.0	0.0	137.0	40.0	35.0	168.0	43.1	2.288

5 rows × 34 columns

Slicing

we use will our diabetes data data,dataset3

In [62]:
```
data3.head(10)
```

Out[62]:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	A
0	9.0	140.0	71.0	32.0	1.0	26.5	0.005	63
1	1.0	85.0	66.0	29.0	0.0	26.6	0.351	3
2	8.0	183.0	64.0	0.0	0.0	23.3	0.672	32
3	1.0	89.0	66.0	23.0	94.0	28.1	0.167	2
4	0.0	137.0	40.0	35.0	168.0	43.1	2.288	33
5	5.0	116.0	74.0	0.0	0.0	25.6	0.201	3(
6	3.0	78.0	50.0	32.0	88.0	31.0	0.248	26
7	10.0	115.0	0.0	0.0	0.0	35.3	0.134	29
8	2.0	197.0	70.0	45.0	543.0	30.5	0.158	53
9	8.0	125.0	96.0	0.0	0.0	0.0	0.232	54

In [65]:
```
data3.iloc[0:2,:]# select rows with index 0,1 not including 2,all select all columns
```

Out[65]:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	A
0	9.0	140.0	71.0	32.0	1.0	26.5	0.005	63
1	1.0	85.0	66.0	29.0	0.0	26.6	0.351	3

In [66]:
```
data3.iloc[:,1:3]# all rows,column index 1-3 not including 3
```

Out[66]:

	Glucose	BloodPressure
0	140.0	71.0
1	85.0	66.0
2	183.0	64.0
3	89.0	66.0
4	137.0	40.0
...	...	...
106	96.0	122.0
107	144.0	58.0
108	190.0	99.0
109	190.0	99.0
110	190.0	99.0

111 rows × 2 columns

In [67]:
```
data3.iloc[1:2,1:3]
```

Out[67]:

	Glucose	BloodPressure
1	85.0	66.0

In [68]:
```
data4.iloc[: , :]
```

Out[68]:

vinesample	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates
45.0	4.2	0.520	0.15	2.1	0.054	8.0	65.0	0.99340	3.90	0.56
95.0	4.7	0.600	0.17	2.3	0.058	17.0	106.0	0.99320	3.85	0.60
821.0	4.9	0.420	0.00	2.1	0.048	16.0	42.0	0.99154	3.71	0.74
588.0	5.0	0.420	0.24	2.0	0.060	19.0	50.0	0.99170	3.72	0.74
94.0	5.0	1.020	0.04	1.4	0.045	41.0	85.0	0.99380	3.75	0.48
...	...	...	...	...	...	...	...	...	...	...
554.0	15.5	0.645	0.49	4.2	0.095	10.0	23.0	1.00315	2.92	0.74
442.0	15.6	0.685	0.76	3.7	0.100	6.0	43.0	1.00320	2.95	0.68
557.0	15.6	0.645	0.49	4.2	0.095	10.0	23.0	1.00315	2.92	0.74
652.0	15.9	0.360	0.65	7.5	0.096	22.0	71.0	0.99760	2.98	0.84
4.6	4.2	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN

1600 rows × 12 columns

In [69]:
```
data4.iloc[2: , :5]
```

Out[69]:

vinesample	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides
821.0	4.9	0.420	0.00	2.1	0.048
588.0	5.0	0.420	0.24	2.0	0.060
94.0	5.0	1.020	0.04	1.4	0.045
553.0	5.0	1.040	0.24	1.6	0.050
1270.0	5.0	0.380	0.01	1.6	0.048
...	...	...	...	...	...
554.0	15.5	0.645	0.49	4.2	0.095
442.0	15.6	0.685	0.76	3.7	0.100
557.0	15.6	0.645	0.49	4.2	0.095
652.0	15.9	0.360	0.65	7.5	0.096
4.6	4.2	NaN	NaN	NaN	NaN

1598 rows × 5 columns

In [70]:
```
data4.iloc[0:5:2 , 2:4:2]     #iloc[starting row:ending row:stepSize , starting column:ending column:st
```

Out[70]:

vinesample	citric acid
45.0	0.15
821.0	0.00
94.0	0.04

merging

In [71]:
```
import pandas as pd
# Define a dictionary containing employee data
data1 = {'key': ['K0', 'K1', 'K2', 'K3'],
 'Name':['Jai', 'Princi', 'Gaurav', 'Anuj'],
 'Age':[27, 24, 22, 32],}

# Define a dictionary containing employee data
data2 = {'key': ['K0', 'K1', 'K2', 'K3'],
 'Address':['Nagpur', 'Kanpur', 'Allahabad', 'Kannuaj'],
 'Qualification':['Btech', 'B.A', 'Bcom', 'B.hons']}
# Convert the dictionary into DataFrame
df = pd.DataFrame(data1)
# Convert the dictionary into DataFrame
df1 = pd.DataFrame(data2)

print(df, "\n\n", df1)
# using .merge() function
res = pd.merge(df, df1, on='key')
res
```

```
  key    Name  Age
0  K0     Jai   27
1  K1  Princi   24
2  K2  Gaurav   22
3  K3    Anuj   32

   key    Address Qualification
0  K0     Nagpur         Btech
1  K1     Kanpur           B.A
2  K2  Allahabad          Bcom
3  K3    Kannuaj        B.hons
```

Out[71]:

	key	Name	Age	Address	Qualification
0	K0	Jai	27	Nagpur	Btech
1	K1	Princi	24	Kanpur	B.A
2	K2	Gaurav	22	Allahabad	Bcom
3	K3	Anuj	32	Kannuaj	B.hons

In [72]:
```
import pandas as pd
# Define a dictionary containing employee data
data1 = {'key': ['K0', 'K1', 'K2', 'K3'],
 'key1': ['K0', 'K1', 'K0', 'K1'],
 'Name':['Jai', 'Princi', 'Gaurav', 'Anuj'],
 'Age':[27, 24, 22, 32],}

# Define a dictionary containing employee data
data2 = {'key': ['K0', 'K1', 'K2', 'K3'],
 'key1': ['K0', 'K0', 'K0', 'K0'],
 'Address':['Nagpur', 'Kanpur', 'Allahabad', 'Kannuaj'],
 'Qualification':['Btech', 'B.A', 'Bcom', 'B.hons']}
# Convert the dictionary into DataFrame
df = pd.DataFrame(data1)
# Convert the dictionary into DataFrame
df1 = pd.DataFrame(data2)
# merging dataframe using multiple keys
res1 = pd.merge(df, df1, on=['key', 'key1'])
res1
```

Out[72]:

	key	key1	Name	Age	Address	Qualification
0	K0	K0	Jai	27	Nagpur	Btech
1	K2	K0	Gaurav	22	Allahabad	Bcom

In [73]:
```
res2 = pd.merge(df, df1, how='outer', on=['key', 'key1'])
res2
```

Out[73]:

	key	key1	Name	Age	Address	Qualification
0	K0	K0	Jai	27.0	Nagpur	Btech
1	K1	K1	Princi	24.0	NaN	NaN
2	K2	K0	Gaurav	22.0	Allahabad	Bcom
3	K3	K1	Anuj	32.0	NaN	NaN
4	K1	K0	NaN	NaN	Kanpur	B.A
5	K3	K0	NaN	NaN	Kannuaj	B.hons

In [74]:
```
# getting intersection of keys
res3 = pd.merge(df, df1, how='inner', on=['key', 'key1'])
res3
```

Out[74]:

	key	key1	Name	Age	Address	Qualification
0	K0	K0	Jai	27	Nagpur	Btech
1	K2	K0	Gaurav	22	Allahabad	Bcom

Working with missing data

we use dataset 4

In [75]:
```
# using isnull() function
data4.isnull()#Finds NULL values
```

Out[75]:

vinesample	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates
45.0	False	False	False	False	False	False	False	False	False	False
95.0	False	False	False	False	False	False	False	False	False	False
821.0	False	False	False	False	False	False	False	False	False	False
588.0	False	False	False	False	False	False	False	False	False	False
94.0	False	False	False	False	False	False	False	False	False	False
...	...	...	...	...	...	...	...	...	...	...
554.0	False	False	False	False	False	False	False	False	False	False
442.0	False	False	False	False	False	False	False	False	False	False
557.0	False	False	False	False	False	False	False	False	False	False
652.0	False	False	False	False	False	False	False	False	False	False
4.6	False	True	True	True	True	True	True	True	True	True

1600 rows × 12 columns

In [76]:
```
# filling missing value using fillna()
data4.fillna(0)
```

Out[76]:

vinesample	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates
45.0	4.2	0.520	0.15	2.1	0.054	8.0	65.0	0.99340	3.90	0.56
95.0	4.7	0.600	0.17	2.3	0.058	17.0	106.0	0.99320	3.85	0.60
821.0	4.9	0.420	0.00	2.1	0.048	16.0	42.0	0.99154	3.71	0.74
588.0	5.0	0.420	0.24	2.0	0.060	19.0	50.0	0.99170	3.72	0.74
94.0	5.0	1.020	0.04	1.4	0.045	41.0	85.0	0.99380	3.75	0.48
...	...	...	...	...	...	...	...	...	...	...
554.0	15.5	0.645	0.49	4.2	0.095	10.0	23.0	1.00315	2.92	0.74
442.0	15.6	0.685	0.76	3.7	0.100	6.0	43.0	1.00320	2.95	0.68
557.0	15.6	0.645	0.49	4.2	0.095	10.0	23.0	1.00315	2.92	0.74
652.0	15.9	0.360	0.65	7.5	0.096	22.0	71.0	0.99760	2.98	0.84
4.6	4.2	0.000	0.00	0.0	0.000	0.0	0.0	0.00000	0.00	0.00

1600 rows × 12 columns

In [77]:
```
# drop missing value
data4.dropna(axis=1)#Drops all rows that contains NULL values
```

Out[77]:

vinesample	fixed acidity
45.0	4.2
95.0	4.7
821.0	4.9
588.0	5.0
94.0	5.0
...	...
554.0	15.5
442.0	15.6
557.0	15.6
652.0	15.9
4.6	4.2

1600 rows × 1 columns

Describe()

The describe() method returns description of the data in the DataFrame.

If the DataFrame contains numerical data, the description contains these information for each column:

count - The number of not-empty values. mean - The average (mean) value. std - The standard deviation. min - the minimum value. 25% - The 25% percentile. *50% - The 50% percentile*. 75% - The 75% percentile*. max - the maximum value.

In [78]:
```
data4.describe()
```

Out[78]:

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide
count	1600.000000	1599.000000	1599.000000	1599.000000	1599.000000	1599.000000	1599.000000
mean	8.316812	0.527821	0.270976	2.538806	0.087467	15.874922	46.467792
std	1.744158	0.179060	0.194801	1.409928	0.047065	10.460157	32.895324
min	4.200000	0.120000	0.000000	0.900000	0.012000	1.000000	6.000000
25%	7.100000	0.390000	0.090000	1.900000	0.070000	7.000000	22.000000
50%	7.900000	0.520000	0.260000	2.200000	0.079000	14.000000	38.000000
75%	9.200000	0.640000	0.420000	2.600000	0.090000	21.000000	62.000000
max	15.900000	1.580000	1.000000	15.500000	0.611000	72.000000	289.000000

Cleaning Data

In [88]:
```
import pandas as pd
data5=pd.read_csv('Social_Network_Ads.csv')
data5
```

Out[88]:

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0
...	...	...	...	...	...
395	15691863	Female	46	41000	1
396	15706071	Male	51	23000	1
397	15654296	Female	50	20000	1
398	15755018	Male	36	33000	0
399	15594041	Female	49	36000	1

400 rows × 5 columns

In [89]:
```
data5.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 400 entries, 0 to 399
Data columns (total 5 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   User ID          400 non-null    int64
 1   Gender           400 non-null    object
 2   Age              400 non-null    int64
 3   EstimatedSalary  400 non-null    int64
 4   Purchased        400 non-null    int64
dtypes: int64(4), object(1)
memory usage: 15.8+ KB
```

In [90]:
```
#drop columns having all values as null values
data5 = data5.dropna(axis=1,how='all')
data5
```

Out[90]:

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0
...	...	...	...	...	...
395	15691863	Female	46	41000	1
396	15706071	Male	51	23000	1
397	15654296	Female	50	20000	1
398	15755018	Male	36	33000	0
399	15594041	Female	49	36000	1

400 rows × 5 columns

In [91]:
```
#replacing male and female with 0 and 1
data5['Gender'].replace({'Female' : 0, 'Male' : 1}, inplace = True)
data5
```

Out[91]:

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	1	19	19000	0
1	15810944	1	35	20000	0
2	15668575	0	26	43000	0
3	15603246	0	27	57000	0
4	15804002	1	19	76000	0
...	...	...	...	...	...
395	15691863	0	46	41000	1
396	15706071	1	51	23000	1
397	15654296	0	50	20000	1
398	15755018	1	36	33000	0
399	15594041	0	49	36000	1

400 rows × 5 columns

In [92]:
```
#converting salary column to integer type
data5['EstimatedSalary']=data5.EstimatedSalary.astype('float64')
data5
```

Out[92]:

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	1	19	19000.0	0
1	15810944	1	35	20000.0	0
2	15668575	0	26	43000.0	0
3	15603246	0	27	57000.0	0
4	15804002	1	19	76000.0	0
...	...	...	...	...	...
395	15691863	0	46	41000.0	1
396	15706071	1	51	23000.0	1
397	15654296	0	50	20000.0	1
398	15755018	1	36	33000.0	0
399	15594041	0	49	36000.0	1

400 rows × 5 columns

In [93]:
```
#The fillna() method allows us to replace empty cells with a value:
data5.fillna(130, inplace = True)
data5
```

Out[93]:

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	1	19	19000.0	0
1	15810944	1	35	20000.0	0
2	15668575	0	26	43000.0	0
3	15603246	0	27	57000.0	0
4	15804002	1	19	76000.0	0
...	...	...	...	...	...
395	15691863	0	46	41000.0	1
396	15706071	1	51	23000.0	1
397	15654296	0	50	20000.0	1
398	15755018	1	36	33000.0	0
399	15594041	0	49	36000.0	1

400 rows × 5 columns

In [94]:
```
#The duplicated() method returns a Boolean values for each row:
#Returns True for every row that is a duplicate, othwerwise False:
print(data5.duplicated())
```

```
0      False
1      False
2      False
3      False
4      False
       ...
395    False
396    False
397    False
398    False
399    False
Length: 400, dtype: bool
```

In [96]:
```
#To remove duplicates, use the drop_duplicates() method.
data5.drop_duplicates(inplace = True)
data5
```

Out[96]:

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	1	19	19000.0	0
1	15810944	1	35	20000.0	0
2	15668575	0	26	43000.0	0
3	15603246	0	27	57000.0	0
4	15804002	1	19	76000.0	0
...	...	...	...	...	...
395	15691863	0	46	41000.0	1
396	15706071	1	51	23000.0	1
397	15654296	0	50	20000.0	1
398	15755018	1	36	33000.0	0
399	15594041	0	49	36000.0	1

400 rows × 5 columns

Pandas Plotting

In [97]:
```
import pandas as pd
data6=pd.read_csv('data.csv')
data6
```

Out[97]:

	Duration	Pulse	Maxpulse	Calories
0	60	110	130	409.1
1	60	117	145	479.0
2	60	103	135	340.0
3	45	109	175	282.4
4	45	117	148	406.0
...	...	...	...	...
164	60	105	140	290.8
165	60	110	145	300.0
166	60	115	145	310.2
167	75	120	150	320.4
168	75	125	150	330.4

169 rows × 4 columns

In [98]:
```
data6.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 169 entries, 0 to 168
Data columns (total 4 columns):
 #   Column    Non-Null Count  Dtype  
---  ------    --------------  -----  
 0   Duration  169 non-null    int64  
 1   Pulse     169 non-null    int64  
 2   Maxpulse  169 non-null    int64  
 3   Calories  164 non-null    float64
dtypes: float64(1), int64(3)
memory usage: 5.4 KB
```

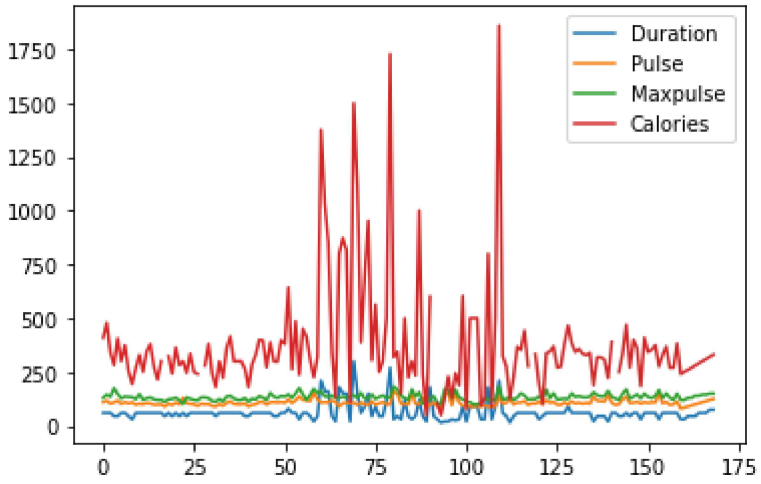
In [99]:
```
data6.describe()
```

Out[99]:

	Duration	Pulse	Maxpulse	Calories
count	169.000000	169.000000	169.000000	164.000000
mean	63.846154	107.461538	134.047337	375.790244
std	42.299949	14.510259	16.450434	266.379919
min	15.000000	80.000000	100.000000	50.300000
25%	45.000000	100.000000	124.000000	250.925000
50%	60.000000	105.000000	131.000000	318.600000
75%	60.000000	111.000000	141.000000	387.600000
max	300.000000	159.000000	184.000000	1860.400000

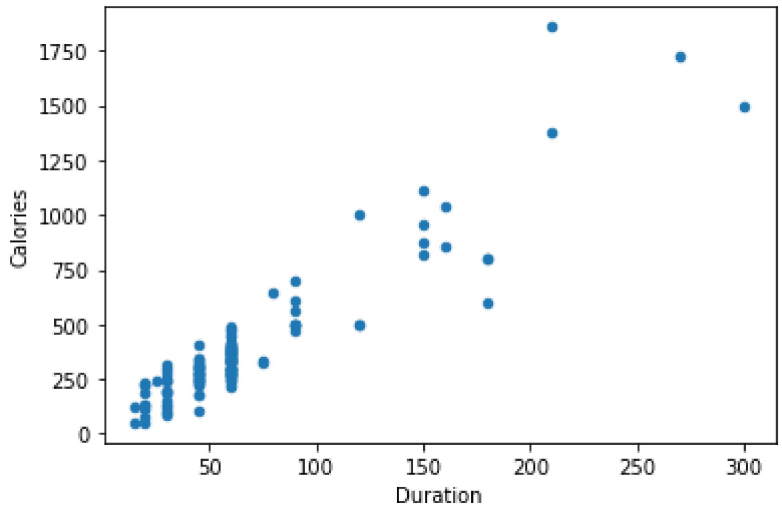
In [100]:
```
import matplotlib.pyplot as plt
data6.plot()

plt.show()
```



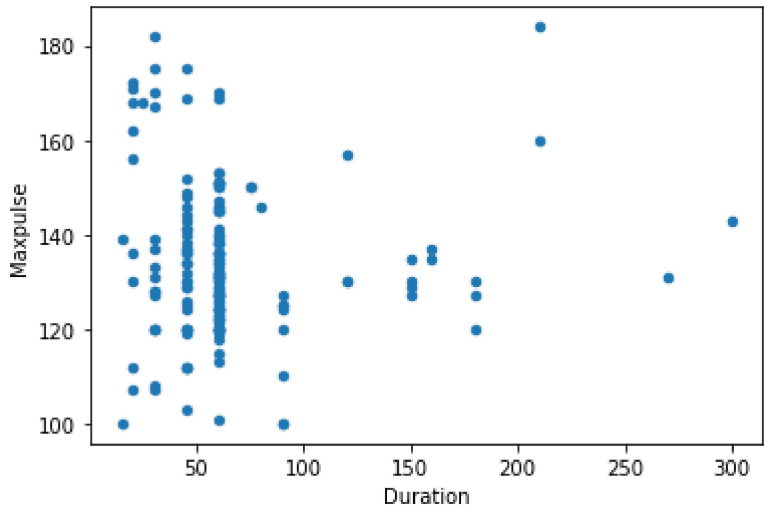
In [101]:
```
data6.plot(kind = 'scatter', x = 'Duration', y = 'Calories')

plt.show()
```



In [102]:
```
data6.plot(kind = 'scatter', x = 'Duration', y = 'Maxpulse')

plt.show()
```



Use the kind argument to specify that you want a histogram:

kind = 'hist'

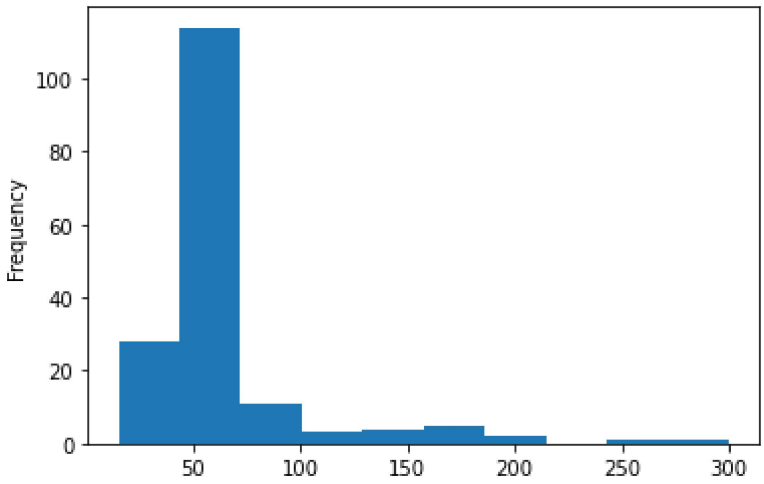
A histogram needs only one column.

A histogram shows us the frequency of each interval, e.g. how many workouts lasted between 50 and 60 minutes?

In the example below we will use the "Duration" column to create the histogram

In [103]:
```
data6["Duration"].plot(kind = 'hist')
```

Out[103]:
```
<matplotlib.axes._subplots.AxesSubplot at 0x26015c0d6d0>
```



Playing with Data Frames

In [1]:
```
import pandas as pd
rd=pd.read_csv('employees.csv')
rd
```

Out[1]:

	First Name	Gender	Start Date	Last Login Time	Salary	Bonus %	Senior Management	Team
0	Douglas	Male	8/6/1993	12:42 PM	97308	6.945	True	Marketing
1	Thomas	Male	3/31/1996	6:53 AM	61933	4.170	True	NaN
2	Maria	Female	4/23/1993	11:17 AM	130590	11.858	False	Finance
3	Jerry	Male	3/4/2005	1:00 PM	138705	9.340	True	Finance
4	Larry	Male	1/24/1998	4:47 PM	101004	1.389	True	Client Services
...	...	...	...	...	...	...	...	...
995	Henry	NaN	11/23/2014	6:09 AM	132483	16.655	False	Distribution
996	Phillip	Male	1/31/1984	6:30 AM	42392	19.675	False	Finance
997	Russell	Male	5/20/2013	12:39 PM	96914	1.421	False	Product
998	Larry	Male	4/20/2013	4:45 PM	60500	11.985	False	Business Development
999	Albert	Male	5/15/2012	6:24 PM	129949	10.169	True	Sales

1000 rows × 8 columns

In [2]:
```
rd.head()
```

Out[2]:

	First Name	Gender	Start Date	Last Login Time	Salary	Bonus %	Senior Management	Team
0	Douglas	Male	8/6/1993	12:42 PM	97308	6.945	True	Marketing
1	Thomas	Male	3/31/1996	6:53 AM	61933	4.170	True	NaN
2	Maria	Female	4/23/1993	11:17 AM	130590	11.858	False	Finance
3	Jerry	Male	3/4/2005	1:00 PM	138705	9.340	True	Finance
4	Larry	Male	1/24/1998	4:47 PM	101004	1.389	True	Client Services

In [3]:
```
rd.iloc[0]
```

Out[3]:
```
First Name              Douglas
Gender                     Male
Start Date             8/6/1993
Last Login Time        12:42 PM
Salary                    97308
Bonus %                   6.945
Senior Management          True
Team                  Marketing
Name: 0, dtype: object
```

In [4]:
```
rd2=pd.read_csv('employees.csv')
rd2=rd2[rd2['Team']=='Marketing']
rd2
```

Out[4]:

	First Name	Gender	Start Date	Last Login Time	Salary	Bonus %	Senior Management	Team
0	Douglas	Male	8/6/1993	12:42 PM	97308	6.945	True	Marketing
21	Matthew	Male	9/5/1995	2:12 AM	100612	13.645	False	Marketing
26	Craig	Male	2/27/2000	7:45 AM	37598	7.757	True	Marketing
43	Marilyn	Female	12/7/1980	3:16 AM	73524	5.207	True	Marketing
62	NaN	Female	6/12/2007	5:25 PM	58112	19.414	NaN	Marketing
...	...	...	...	...	...	...	...	...
942	Lori	Female	11/20/2015	1:15 PM	75498	6.537	True	Marketing
947	NaN	Male	7/30/2012	3:07 PM	107351	5.329	NaN	Marketing
986	Donna	Female	11/26/1982	7:04 AM	82871	17.999	False	Marketing
991	Rose	Female	8/25/2002	5:12 AM	134505	11.051	True	Marketing
994	George	Male	6/21/2013	5:47 PM	98874	4.479	True	Marketing

98 rows × 8 columns

Multindexing in Python

In [5]:
```
import pandas as pd
rd3=pd.read_csv('employees.csv')
rd3.head()
```

Out[5]:

	First Name	Gender	Start Date	Last Login Time	Salary	Bonus %	Senior Management	Team
0	Douglas	Male	8/6/1993	12:42 PM	97308	6.945	True	Marketing
1	Thomas	Male	3/31/1996	6:53 AM	61933	4.170	True	NaN
2	Maria	Female	4/23/1993	11:17 AM	130590	11.858	False	Finance
3	Jerry	Male	3/4/2005	1:00 PM	138705	9.340	True	Finance
4	Larry	Male	1/24/1998	4:47 PM	101004	1.389	True	Client Services

In [9]:
```
rd3.set_index(['Team','Salary'],inplace=True)
```

In [10]:
```
rd3.loc['Finance']
```

Out[10]:

Salary	First Name	Gender	Start Date	Last Login Time	Bonus %	Senior Management
130590	Maria	Female	4/23/1993	11:17 AM	11.858	False
138705	Jerry	Male	3/4/2005	1:00 PM	9.340	True
45906	NaN	Female	7/20/2015	10:43 AM	11.598	NaN
41426	Kimberly	Female	1/14/1999	7:13 AM	14.543	True
114796	Bruce	Male	11/28/2009	10:47 PM	6.796	False
...	...	...	...	...	...	...
137144	Elizabeth	Female	7/27/1998	11:12 AM	10.081	False
119667	Joe	Male	1/19/1980	4:06 PM	1.148	True
136709	Gloria	Female	12/8/2014	5:08 AM	10.331	True
112769	Anthony	Male	10/16/2011	8:35 AM	11.625	True
42392	Phillip	Male	1/31/1984	6:30 AM	19.675	False

102 rows × 6 columns

In [12]:
```
rd3.loc['Finance'].loc[130590]
```

Out[12]:
```
First Name              Maria
Gender                 Female
Start Date           4/23/1993
Last Login Time       11:17 AM
Bonus %                 11.858
Senior Management        False
Name: 130590, dtype: object
```

In [4]:
```
import pandas as pd
test=pd.read_csv('employees.csv')
test.head()
```

Out[4]:

	First Name	Gender	Start Date	Last Login Time	Salary	Bonus %	Senior Management	Team
0	Douglas	Male	8/6/1993	12:42 PM	97308	6.945	True	Marketing
1	Thomas	Male	3/31/1996	6:53 AM	61933	4.170	True	NaN
2	Maria	Female	4/23/1993	11:17 AM	130590	11.858	False	Finance
3	Jerry	Male	3/4/2005	1:00 PM	138705	9.340	True	Finance
4	Larry	Male	1/24/1998	4:47 PM	101004	1.389	True	Client Services

In [5]:
```
test.columns
```

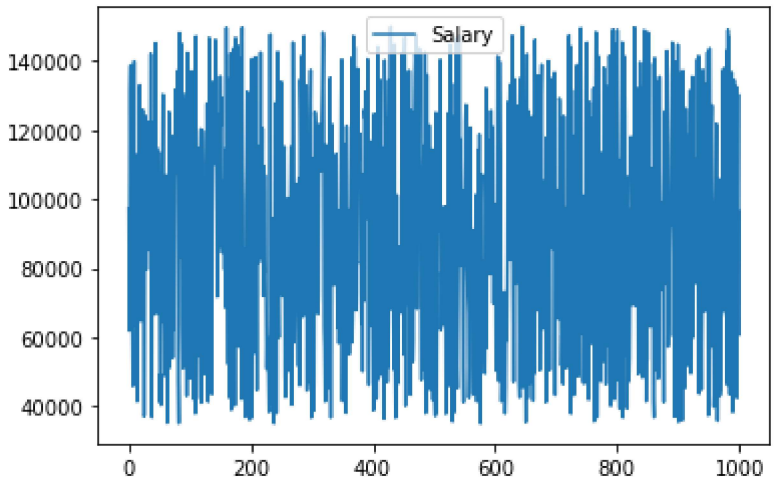
Out[5]:
```
Index(['First Name', 'Gender', 'Start Date', 'Last Login Time', 'Salary',
       'Bonus %', 'Senior Management', 'Team'],
      dtype='object')
```

In [6]:
```
test.plot()
```

Out[6]:
```
<matplotlib.axes._subplots.AxesSubplot at 0x234bec26bb0>
```

In [9]:
```
test['Salary'].plot(legend=True)
```

Out[9]:
```
<matplotlib.axes._subplots.AxesSubplot at 0x234bf476e50>
```



In [2]:
```
# test.plot(x='Salary',y='Bonus%',kind='scatter',legend=True)
```

In [ ]:
```
# test['Salary'].plot(kind='box',title='box plot')
```